

进一步讲,如果类包含 `vector` 或者 `string` 成员,则其拷贝、赋值和销毁的合成版本能够正常工作。当我们对含有 `vector` 成员的对象执行拷贝或者赋值操作时, `vector` 类会设法拷贝或者赋值成员中的元素。当这样的对象被销毁时,将销毁 `vector` 对象,也就是依次销毁 `vector` 中的每一个元素。这一点与 `string` 是非常类似的。



在学习第 13 章关于如何自定义操作的知识之前,类中所有分配的资源都应该直接以类的数据成员的形式存储。



7.2 访问控制与封装

268

到目前为止,我们已经为类定义了接口,但并没有任何机制强制用户使用这些接口。我们的类还没有封装,也就是说,用户可以直达 `Sales_data` 对象的内部并且控制它的具体实现细节。在 C++ 语言中,我们使用访问说明符(access specifiers)加强类的封装性:

- 定义在 **public** 说明符之后的成员在整个程序内可被访问, `public` 成员定义类的接口。
- 定义在 **private** 说明符之后的成员可以被类的成员函数访问,但是不能被使用该类的代码访问, `private` 部分封装了(即隐藏了)类的实现细节。

再一次定义 `Sales_data` 类,其新形式如下所示:

```
class Sales_data {
public:           // 添加了访问说明符
    Sales_data() = default;
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    Sales_data(const std::string &s): bookNo(s) { }
    Sales_data(std::istream&);
    std::string isbn() const { return bookNo; }
    Sales_data &combine(const Sales_data&);
private:        // 添加了访问说明符
    double avg_price() const
        { return units_sold ? revenue/units_sold : 0; }
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
```

作为接口的一部分,构造函数和部分成员函数(即 `isbn` 和 `combine`)紧跟在 `public` 说明符之后;而数据成员和作为实现部分的函数则跟在 `private` 说明符后面。

一个类可以包含 0 个或多个访问说明符,而且对于某个访问说明符能出现多少次也没有严格限定。每个访问说明符指定了接下来的成员的访问级别,其有效范围直到出现下一个访问说明符或者到达类的结尾处为止。

使用 `class` 或 `struct` 关键字

在上面的定义中我们还做了一个微妙的变化,我们使用了 **class** 关键字而非 **struct** 开始类的定义。这种变化仅仅是形式上有所不同,实际上我们可以使用这两个关键字中的任何一个定义类。唯一的一点区别是, `struct` 和 `class` 的默认访问权限不太一样。

类可以在它的第一个访问说明符之前定义成员,对这种成员的访问权限依赖于类定义

的方式。如果我们使用 `struct` 关键字,则定义在第一个访问说明符之前的成员是 `public` 的;相反,如果我们使用 `class` 关键字,则这些成员是 `private` 的。

出于统一编程风格的考虑,当我们希望定义的类的所有成员是 `public` 的时,使用 `struct`;反之,如果希望成员是 `private` 的,使用 `class`。



使用 `class` 和 `struct` 定义类唯一的区别就是默认访问权限。

7.2 节练习

练习 7.16: 在类的定义中对于访问说明符出现的位置和次数有限定吗?如果有,是什么?什么样的成员应该定义在 `public` 说明符之后?什么样的成员应该定义在 `private` 说明符之后?

练习 7.17: 使用 `class` 和 `struct` 时有区别吗?如果有,是什么?

练习 7.18: 封装是何含义?它有什么用处?

练习 7.19: 在你的 `Person` 类中,你将把哪些成员声明成 `public` 的?哪些声明成 `private` 的?解释你这样做的原因。

7.2.1 友元



既然 `Sales_data` 的数据成员是 `private` 的,我们的 `read`、`print` 和 `add` 函数也就无法正常编译了,这是因为尽管这几个函数是类的接口的一部分,但它们不是类的成员。

类可以允许其他类或者函数访问它的非公有成员,方法是令其他类或者函数成为它的友元(friend)。如果类想把一个函数作为它的友元,只需要增加一条以 `friend` 关键字开始的函数声明语句即可:

```
class Sales_data {
// 为 Sales_data 的非成员函数所做的友元声明
friend Sales_data add(const Sales_data&, const Sales_data&);
friend std::istream &read(std::istream&, Sales_data&);
friend std::ostream &print(std::ostream&, const Sales_data&);
// 其他成员及访问说明符与之前一致
public:
    Sales_data() = default;
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    Sales_data(const std::string &s): bookNo(s) { }
    Sales_data(std::istream&);
    std::string isbn() const { return bookNo; }
    Sales_data &combine(const Sales_data&);
private:
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
// Sales_data 接口的非成员组成部分的声明
Sales_data add(const Sales_data&, const Sales_data&);
```

```
std::istream &read(std::istream&, Sales_data&);
std::ostream &print(std::ostream&, const Sales_data&);
```

友元声明只能出现在类定义的内部，但是在类内出现的具体位置不限。友元不是类的成员，也不受它所在区域访问控制级别的约束。我们将在 7.3.4 节（第 250 页）介绍更多关于友元的知识。



一般来说，最好在类定义开始或结束前的位置集中声明友元。

关键概念：封装的益处

封装有两个重要的优点：

- 确保用户代码不会无意间破坏封装对象的状态。
- 被封装的类的具体实现细节可以随时改变，而无须调整用户级别的代码。

一旦把数据成员定义成 `private` 的，类的作者就可以比较自由地修改数据了。当实现部分改变时，我们只需要检查类的代码本身以确认这次改变有什么影响；换句话说，只要类的接口不变，用户代码就无须改变。如果数据是 `public` 的，则所有使用了原来数据成员的代码都可能失效，这时我们必须定位并重写所有依赖于老版本实现的代码，之后才能重新使用该程序。

把数据成员的访问权限设成 `private` 还有另外一个好处，这么做能防止由于用户的原因造成数据被破坏。如果我们发现有程序缺陷破坏了对象的状态，则可以在有限的范围内定位缺陷：因为只有实现部分的代码可能产生这样的错误。因此，将查错限制在有限范围内将能极大地降低维护代码及修正程序错误的难度。



尽管当类的定义发生改变时无须更改用户代码，但是使用了该类的源文件必须重新编译。



友元的声明

友元的声明仅仅指定了访问的权限，而非一个通常意义上的函数声明。如果我们希望类的用户能够调用某个友元函数，那么我们就必须在友元声明之外再专门对函数进行一次声明。

为了使友元对类的用户可见，我们通常把友元的声明与类本身放置在同一个头文件中（类的外部）。因此，我们的 `Sales_data` 头文件应该为 `read`、`print` 和 `add` 提供独立的声明（除了类内部的友元声明之外）。



许多编译器并未强制限定友元函数必须在使用之前在类的外部声明。

271

一些编译器允许在尚无友元函数的初始声明的情况下就调用它。不过即使你的编译器支持这种行为，最好还是提供一个独立的函数声明。这样即使你更换了一个有这种强制要求的编译器，也不必改变代码。

7.2.1 节练习

练习 7.20: 友元在什么时候有用? 请分别列举出使用友元的利弊。

练习 7.21: 修改你的 `Sales_data` 类使其隐藏实现的细节。你之前编写的关于 `Sales_data` 操作的程序应该继续使用, 借助类的新定义重新编译该程序, 确保其工作正常。

练习 7.22: 修改你的 `Person` 类使其隐藏实现的细节。

7.3 类的其他特性

虽然 `Sales_data` 类非常简单, 但是通过它我们已经了解 C++ 语言中关于类的许多语法要点。在本节中, 我们将继续介绍 `Sales_data` 没有体现出来的一些类的特性。这些特性包括: 类型成员、类的成员的类内初始值、可变数据成员、内联成员函数、从成员函数返回 `*this`、关于如何定义并使用类类型及友元类的更多知识。

7.3.1 类成员再探

为了展示这些新的特性, 我们需要定义一对相互关联的类, 它们分别是 `Screen` 和 `Window_mgr`。

定义一个类型成员

`Screen` 表示显示器中的一个窗口。每个 `Screen` 包含一个用于保存 `Screen` 内容的 `string` 成员和三个 `string::size_type` 类型的成员, 它们分别表示光标的位置以及屏幕的高和宽。

除了定义数据和函数成员之外, 类还可以自定义某种类型在类中的别名。由类定义的类型名字和其他成员一样存在访问限制, 可以是 `public` 或者 `private` 中的一种:

```
class Screen {
public:
    typedef std::string::size_type pos;
private:
    pos cursor = 0;
    pos height = 0, width = 0;
    std::string contents;
};
```

我们在 `Screen` 的 `public` 部分定义了 `pos`, 这样用户就可以使用这个名字。 `Screen` 的用户不应该知道 `Screen` 使用了一个 `string` 对象来存放它的数据, 因此通过把 `pos` 定义成 `public` 成员可以隐藏 `Screen` 实现的细节。 ◀ 272

关于 `pos` 的声明有两点需要注意。首先, 我们使用了 `typedef` (参见 2.5.1 节, 第 60 页), 也可以等价地使用类型别名 (参见 2.5.1 节, 第 60 页):

```
class Screen {
public:
    // 使用类型别名等价地声明一个类型名字
    using pos = std::string::size_type;
    // 其他成员与之前的版本一致
};
```

其次, 用来定义类型的成员必须先定义后使用, 这一点与普通成员有所区别, 具体原因将